

Architecture Document

Local-First Financial Analysis Application — V1

Status: Draft v2 (post-review revision) **Companion to:** docs/prd.md **Audience:** Engineers, technical reviewers, future contributors

Changes since v1: Resolves the architecture-reviewer's critical findings C1–C5 and the applicable major findings. Incorporates the V1 Q&A answers: 8-K Item 4.02 parsing for restatement warnings, market-cap split (historical persisted offline / current online-only), full non-USD-filer support with FX conversion, deterministic YTD-to-quarterly derivation, linked-list supersession chain with cycle protection, period-table fixes for 52/53-week and FYE changes, single-primary normalization model, idempotent refresh via natural-key UNIQUE on `raw_fact`, Tauri 2 capabilities + `request` host allowlist (replacing the loose "Tauri allowlist" wording), and Developer ID-signed `.dmg` distribution with no auto-update for V1.

1. Introduction

1.1 Purpose

This document defines the technical architecture for V1 of a local-first macOS financial analysis application. It translates the product requirements in docs/prd.md into concrete technology choices, module boundaries, data structures, and runtime behavior. It is the source of truth for "how" the system is built; the PRD is the source of truth for "what" it must do.

1.2 Scope

V1 only. The architecture must accommodate the V2/V3 directions called out in the PRD (multi-company comparison, formula engine, export, plugins) without requiring a rewrite, but no V2+ feature is implemented in V1.

1.3 Key Architectural Drivers

Drawn directly from the PRD and ranked:

1. **Correctness** of financial data — the user must be able to trust every number.
2. **Traceability** — every displayed value must trace back to a specific SEC filing concept.
3. **Local-first** — no cloud backend, full offline operation after ingestion.
4. **Durability** — application crashes must not destroy ingested data.
5. **Modularity** — clean seams for future expansion (peer comparison, formula engine, export).
6. **Maintainability** — a small team / single developer should be able to evolve the codebase.
7. **Performance** — ingestion may be slow; navigation of ingested data must feel instant.

Speed of ingestion is explicitly the *lowest* priority among these.

Data acquisition note. V1's primary structured-data source is the SEC's pre-extracted XBRL facts JSON API at `data.sec.gov/api/xbrl/companyfacts`. *General* HTML / iXBRL parsing of 10-K and 10-Q narrative content is out of V1 scope. *Narrow*, accuracy-driven HTML parsing is in scope where it is the only path to a precise result — specifically, parsing 8-K Item 4.02 disclosures to identify the fiscal periods flagged as unreliable, and fetching historical EOD prices for market-cap-at-filing-date computation. See §3.5.

2. Architectural Style

The system is a **local-first, modular monolith** packaged as a single desktop application. There is no client/server split, no microservices, and no remote infrastructure.

Internally the codebase is organized as a layered architecture with a separate **pipeline architecture** for the ingestion subsystem:

- **Layered (UI → IPC → Domain Services → Persistence):** for read-side workflows, where the dashboard queries normalized data through repository interfaces.
- **Pipeline (Discover → Download → Parse → Normalize → Persist):** for the ingestion subsystem, where each stage is an isolated, testable unit with explicit inputs and outputs.

This split is deliberate: the read path optimizes for low-latency UI rendering, while the write path optimizes for correctness, observability, and resumability.

3. Technology Stack

3.1 Decision Summary

Concern	Choice	Alternative considered
Desktop shell	Tauri 2.x	Electron, Swift/AppKit
Backend language	Rust (in Tauri's <code>src-tauri/</code>)	Go, TypeScript (Node)
Frontend	React 18 + TypeScript + Vite	SwiftUI, Svelte
Styling	Tailwind CSS	CSS Modules, Stitches
Charting	Apache ECharts (via <code>echarts-for-react</code>)	Recharts, Chart.js
State / data fetching	TanStack Query over Tauri IPC	Redux, Zustand
Local database	SQLite via <code>rusqlite</code> (bundled)	DuckDB, Postgres
HTTP client	reqwest with <code>rustls</code>	<code>ureq</code> , <code>hyper</code>
Async runtime	tokio	<code>async-std</code>
Logging / tracing	tracing + <code>tracing-subscriber</code>	<code>log</code> + <code>env_logger</code>
Error handling	thiserror (lib) + anyhow (app boundary)	manual enums
Migrations	refinery	hand-rolled, <code>sqlx-migrate</code>
Build / package	Tauri bundler → <code>.dmg</code>	<code>cargo-bundle</code>

3.2 Rationale: Tauri over Electron

- **Bundle size & memory:** Tauri applications are typically ~3–10 MB and use the system WebView (WKWebView on macOS), versus Electron's ~80–150 MB Chromium runtime. For a single-user analytical app, the lighter footprint matters.
- **Security:** Tauri's IPC surface is opt-in per command, with an explicit allowlist. The PRD's "minimize third-party dependencies" and "avoid unnecessary outbound requests" principles map naturally onto Tauri's permission model.

- **Rust backend:** JSON parsing of SEC's `companyfacts` payload, taxonomy normalization, XBRL XML fallback parsing, narrow HTML parsing of 8-K Item 4.02 disclosures, market-data API clients, and SQLite work all benefit from Rust's correctness guarantees, error-handling discipline, and zero-cost abstractions. The normalization subsystem in particular is the kind of code where Rust's type system pays for itself.
- **Native macOS look-and-feel:** WKWebView integrates with macOS conventions (text rendering, scrolling, accessibility) more naturally than bundled Chromium.

3.3 Why not native Swift/AppKit?

- Significantly larger surface area for a single-developer effort.
- The financial-domain code (XBRL parsing, normalization) is not language-specific — keeping it in a portable Rust core preserves the option to ship Windows/Linux builds later without rewriting.
- The PRD explicitly lists Tauri as an acceptable approach.

3.4 Rationale: SQLite over DuckDB

DuckDB is more attractive for ad-hoc analytical queries, but for V1:

- The dataset is small (one company $\times$ $\sim$ 80 quarters $\times$ $\sim$ 200 facts $\approx$ 16 K rows; even 50 companies stays well under 1 M rows).
- SQLite has more mature tooling, ACID guarantees the PRD requires for durability, and a more stable on-disk format for long-term local storage.
- Tauri + `rusqlite` has a well-trodden integration path.
- DuckDB can be added later as a read-only analytical accelerator without changing the canonical store.

3.5 Rationale: how V1 acquires financial data from SEC EDGAR

This is the most important data-flow decision in V1. It is worth being precise about what is and is not on the wire.

10-K and 10-Q documents themselves are not in JSON. A filing is submitted to EDGAR as a bundle of HTML / inline XBRL (iXBRL) / XBRL XML and exhibits. There is no public API endpoint that returns "the 10-K, as JSON." Anyone parsing a 10-K is parsing HTML or XML.

What V1 actually consumes is a different artifact. Since the SEC XBRL mandate (large filers $\sim$ 2009, all U.S.-listed filers $\sim$ 2011), every numeric line item on the financial statements in a 10-K, 10-Q, or financial-statement 8-K must be tagged with a structured XBRL concept (e.g., `us-gaap:Revenues`, `us-gaap:Assets`, `us-gaap:NetIncomeLoss`). The SEC ingests those tagged values into a structured database and exposes a public read-side at:

```
https://data.sec.gov/api/xbrl/companyfacts/CIK{cik10}.json
```

This JSON is **not the 10-K**. It is the SEC's own pre-extracted view of every XBRL fact the company has ever tagged in a 10-K / 10-Q / 8-K, aggregated into a single JSON document keyed by taxonomy and concept. Each fact in the response carries:

Field	Meaning
val	The numeric value, in the declared unit
unit (key)	USD , shares , USD/shares , etc.
start , end	Period covered (or end only, for instant facts)
fy	Fiscal year
fp	Fiscal period (FY , Q1 , Q2 , Q3 , Q4)
form	Source form type (10-K , 10-Q , 10-K/A , 8-K , ...)
accn	Accession number of the source filing

The `accn` is what gives V1 its source-filing traceability for free (FR-060): every normalized value can be linked back to the exact filing it came from without us touching the filing itself.

What V1 gets from `companyfacts` : every tagged numeric line item on the income statement, balance sheet, and cash-flow statement, across every reporting period the company has filed, for any U.S.-listed company that complies with the XBRL mandate. This is exactly the data the V1 PRD requires (FR-020 / FR-021 / FR-022).

What V1 does *not* get from `companyfacts` :

- MD&A narrative
- Footnote prose
- Risk factors and other Reg S-K narrative sections
- Exhibits in their entirety
- Anything inside the 10-K that is not XBRL-tagged

General HTML parsing of 10-K narrative content — to support summarization, sentiment analysis, "what did management say" — is not in V1 scope. The PRD defers that class of capability to V3.

Narrow, accuracy-driven HTML parsing IS in V1 scope. Per the PRD's correctness driver and FR-011 (which explicitly lists "HTML/text parsing fallback" as a permitted source), V1 includes two targeted HTML/text parsing paths. Both are narrow, well-bounded, and motivated by data-integrity rather than feature breadth:

1. **8-K Item 4.02 disclosures** — parsed to identify the specific fiscal periods the company is flagging as unreliable. The system must reliably extract those period identifiers regardless of phrasing or whether the filing carries structured tags. Result is stored in a dedicated `restatement_announcement` table (see §6.3) and used to surface a per-period warning in the dashboard, which clears once amendments covering the named periods are filed. **No financial values are extracted from 8-Ks.** See §6.3, §8.7, §9.1.
2. **Historical EOD prices** for market-cap-at-filing-date computation. Per FR-050, V1 must show historical market cap at each filing date offline; this is computed at ingestion time as $\text{price} \times \text{shares-outstanding-from-companyfacts}$ and persisted locally. Price data comes from a separate market-data adapter (see §7.5), not from EDGAR.

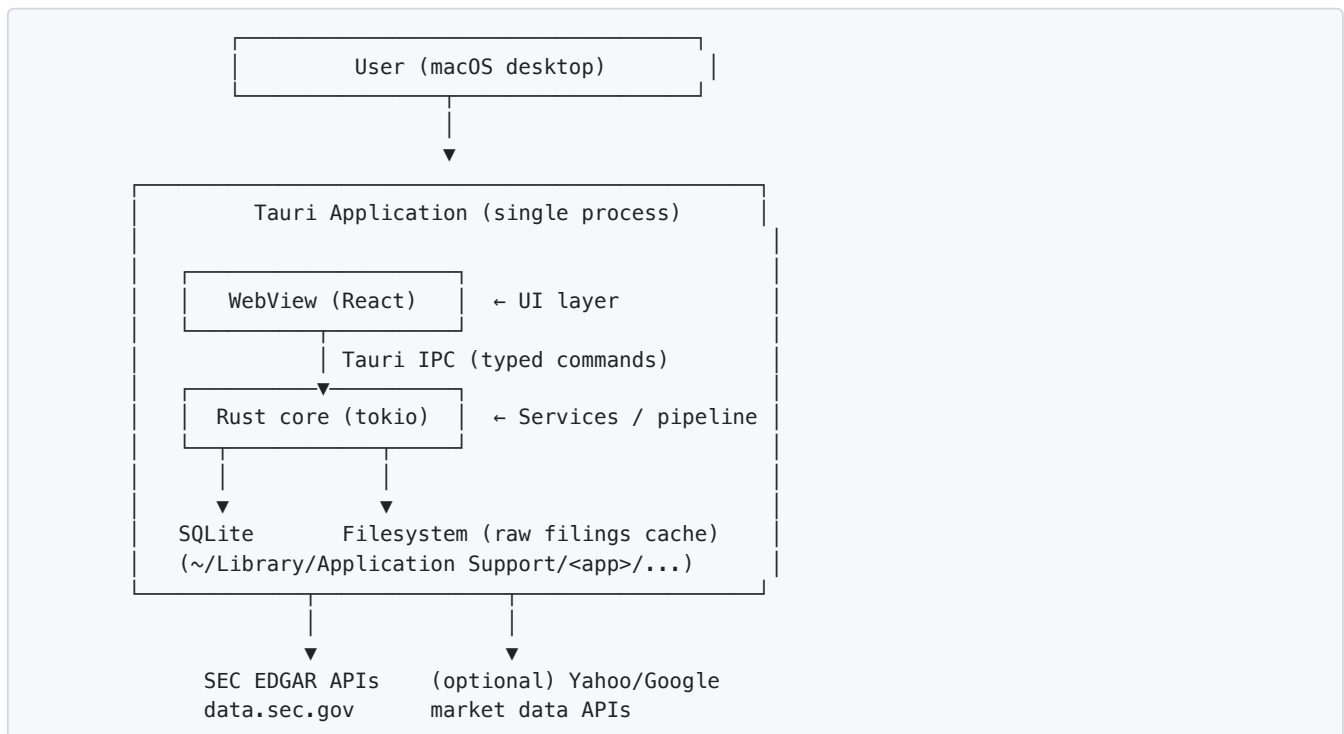
Summary of V1 data sources:

Source	Format	Used for	Path
<code>data.sec.gov/api/xbrl/companyfacts/CIK*.json</code>	JSON	Structured financial facts (primary)	Normal
<code>data.sec.gov/submissions/CIK*.json</code>	JSON	Filing index, including 8-K item lists	Normal
Raw XBRL instance document	XML	Targeted re-fetch of values missing from <code>companyfacts</code>	Fallback
8-K primary document (Item 4.02 only)	HTML / iXBRL	Identifying restated periods	Normal (when an Item 4.02 8-K is in the submissions index)
Market-data adapter	JSON (provider-specific)	Historical EOD prices for market cap	Normal

What is still excluded from V1: parsing 10-K / 10-Q HTML narrative, parsing any 8-K item other than Item 4.02, downloading filing exhibits, and any text-extraction work whose output is not a structured fact backed by full traceability. The fallback is implemented behind the same `FactSource` trait as the JSON path, so it can be enabled per-concept-per-filing without disturbing the rest of the pipeline.

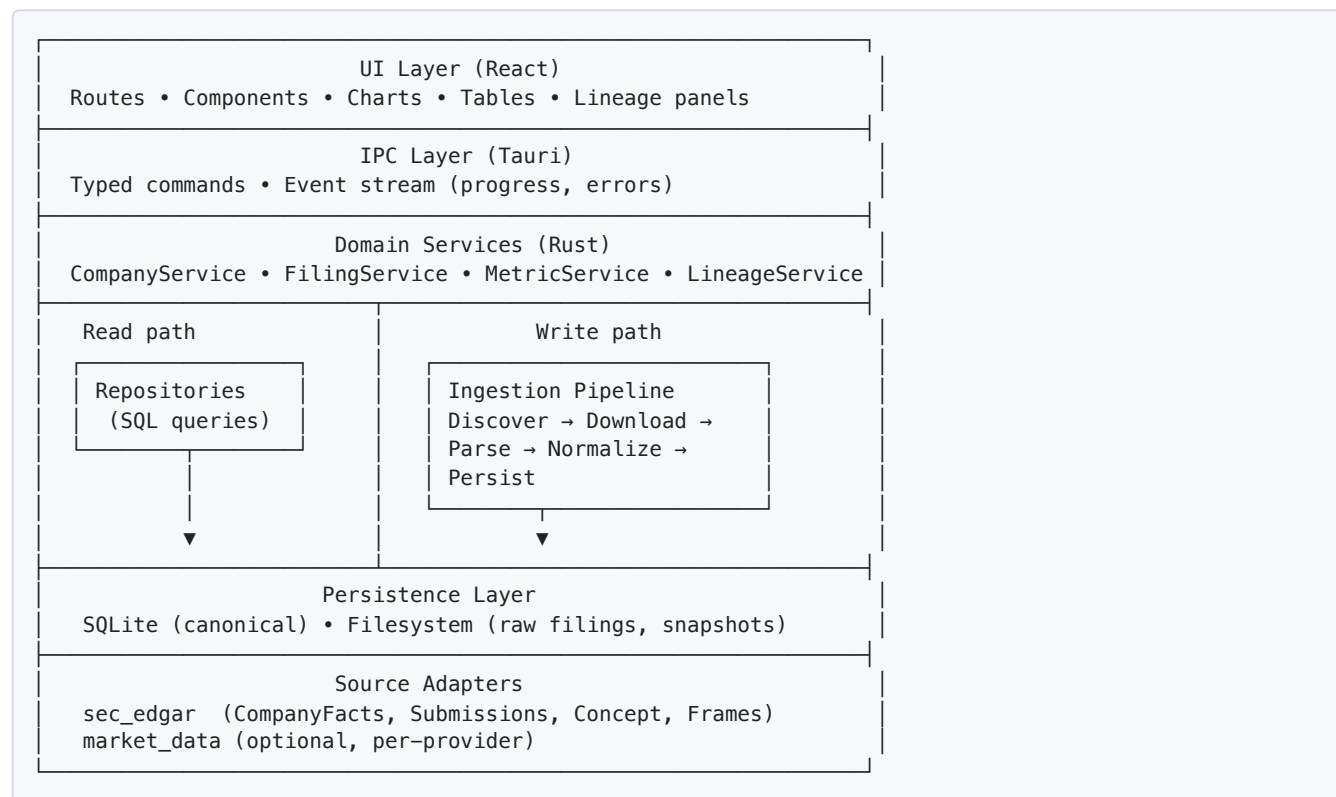
4. High-Level Architecture

4.1 System Context



There is exactly one outbound dependency that is required for ingestion: the SEC EDGAR APIs. Market-data APIs are optional and the application degrades gracefully without them.

4.2 Layer Diagram



The strict downward dependency rule applies: each layer depends only on layers below it. The UI never reaches into the database directly; the ingestion pipeline never calls into the UI; source adapters never know about SQL.

5. Module Structure

5.1 Repository layout

```
econ_project/
├── docs/                # PRD, architecture, future ADRs
├── src/                 # React app
│   ├── api/            # Typed Tauri IPC client
│   ├── features/
│   │   ├── home/       # Saved companies, add ticker
│   │   ├── company/    # Dashboard, charts, tables
│   │   ├── lineage/    # Drill-down to filing source
│   │   └── ingestion/  # Progress UI
│   ├── components/     # Generic UI primitives
│   ├── charts/         # ECharts wrappers
│   ├── state/          # TanStack Query hooks
│   └── styles/
├── src-tauri/          # Rust core
│   ├── src/
│   │   ├── main.rs
│   │   ├── ipc/        # #[tauri::command] entry points
│   │   ├── domain/     # Service layer (use-cases)
│   │   ├── ingestion/  # Pipeline stages
│   │   │   ├── discover.rs
│   │   │   ├── download.rs
│   │   │   ├── parse.rs
│   │   │   ├── normalize.rs
│   │   │   └── persist.rs
│   │   ├── normalization/ # Concept maps, unit/period rules
│   │   │   ├── concept_map.rs
│   │   │   ├── periods.rs
│   │   │   ├── units.rs
│   │   │   └── signs.rs
│   │   ├── sources/
│   │   │   ├── sec_edgar/ # companyfacts/submissions JSON, XBRL XML fallback,
│   │   │   │               # Item 4.02 8-K HTML parser
│   │   │   └── market_data/ # MarketDataAdapter trait + Yahoo Finance default impl
│   │   ├── persistence/
│   │   │   ├── db.rs     # Connection pool, migrations
│   │   │   ├── repositories/ # One module per aggregate
│   │   │   └── schema/   # .sql migrations
│   │   ├── derived/     # Derived-metric formulas
│   │   ├── lineage/     # Source-tracking helpers
│   │   ├── errors.rs
│   │   └── telemetry.rs  # tracing setup (local logs only)
├── Cargo.toml
└── README.md
```

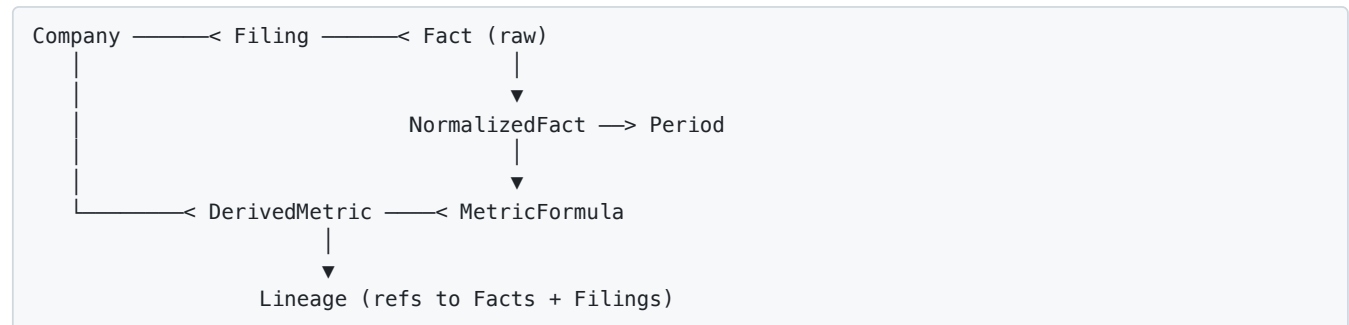
5.2 Crate / module boundaries

The Rust code is one binary crate but organized so that each top-level module under `src-tauri/src/` could be extracted into its own crate later without churn. In particular:

- `normalization/` depends on no I/O, only on domain types — it is pure logic and should be heavily unit-tested.
- `sources/sec_edgar/` depends on `reqwest` and the raw EDGAR JSON shape, but exposes only typed `RawFact` / `RawFiling` records to the rest of the system.
- `persistence/repositories/` is the only module allowed to write SQL.

6. Data Architecture

6.1 Conceptual model



6.2 Canonical metric model

The product's correctness hinges on a stable internal vocabulary. V1 ships a hard-coded **canonical metric catalog** — a small enum of well-defined financial concepts with documented semantics:

Canonical metric	Statement	Aggregation	Origin	Sign convention
revenue	Income	Period flow	XBRL	Positive
cost_of_revenue	Income	Period flow	XBRL	Positive
gross_profit	Income	Period flow	XBRL or derived (revenue - cost_of_revenue)	Positive
operating_income	Income	Period flow	XBRL	Signed
net_income	Income	Period flow	XBRL	Signed
eps_basic	Income	Period flow	XBRL	Signed
eps_diluted	Income	Period flow	XBRL	Signed
shares_outstanding_basic	Income	Period instant	XBRL	Positive
shares_outstanding_diluted	Income	Period instant	XBRL	Positive
cash_and_equivalents	Balance	Instant	XBRL	Positive
long_term_debt	Balance	Instant	XBRL	Positive
current_debt	Balance	Instant	XBRL	Positive
total_debt	Balance	Instant	Derived (long_term_debt + current_debt)	Positive
total_assets	Balance	Instant	XBRL	Positive
total_liabilities	Balance	Instant	XBRL	Positive
total_equity	Balance	Instant	XBRL	Signed
cash_from_operations	Cash flow	Period flow	XBRL	Signed
capital_expenditures	Cash flow	Period flow	XBRL	Stored positive (sign-normalized)
depreciation_amortization	Cash flow	Period flow	XBRL	Positive
historical_market_cap	Market	Instant (at filing date)	Derived (historical_price_at_filed_at × shares_outstanding_basic)	Positive
current_market_cap	Market	Live	Derived (live_price × latest_shares_outstanding_basic)	Positive

Each entry records its statement, whether it is a flow (period) or stock (instant), how it originates, and the canonical sign convention used at storage time. UI display rules can then invert signs consistently for presentation (e.g., showing CapEx as a negative cash outflow on a cash-flow waterfall).

total_debt definition. XBRL has no single canonical "total debt" concept. V1 defines `total_debt = long_term_debt + current_debt`, where the inputs map to a primary-then-fallback chain of XBRL concepts:

- `long_term_debt` ← `us-gaap:LongTermDebt`, falling back to `us-gaap:LongTermDebtNoncurrent` when the primary is absent.
- `current_debt` ← `us-gaap:DebtCurrent`, falling back to `us-gaap:LongTermDebtCurrent`.

The formula and the resolved input concepts are surfaced in the lineage panel for transparency (FR-031).

Market-cap metrics.

- `historical_market_cap` is computed once per filing at ingestion time and persisted. Its inputs are the historical EOD price on the filing's `filed_at` date (from a market-data adapter — see §7.5) and `shares_outstanding_basic` from the same filing. It is offline-available because it is fully persisted.
- `current_market_cap` is computed on demand from a live price source. When the live source is unavailable (offline or the source is down), the dashboard widget renders an explicit "current market cap unavailable" state and the historical series remains visible.

No TTM in V1. An earlier draft proposed a TTM aggregation. TTM is not a PRD requirement, and the PRD's annual/quarterly chart toggle (FR-051) already addresses the long-horizon analytical need. V1 defers TTM; the dashboard summary widgets show the latest reported annual value with a sparkline of recent quarters instead of a synthetic TTM.

The catalog is a Rust enum so missing values are caught at compile time. New metrics are added by extending the enum, the concept map, and (where applicable) a derived formula.

6.3 SQLite schema (V1)

```
-- Companies
CREATE TABLE company (
  cik          TEXT PRIMARY KEY,          -- 10-digit zero-padded
  ticker       TEXT NOT NULL,
  name         TEXT NOT NULL,
  exchange     TEXT,
  sic          TEXT,
  fiscal_year_end TEXT,                  -- MMDD
  added_at     TEXT NOT NULL,           -- ISO8601
  last_refreshed TEXT
);
CREATE UNIQUE INDEX idx_company_ticker ON company(ticker);

-- Filings (one row per accession number per company)
CREATE TABLE filing (
  accession_no  TEXT PRIMARY KEY,        -- e.g. "0000320193-24-000123"
  cik          TEXT NOT NULL REFERENCES company(cik),
  form_type    TEXT NOT NULL,           -- 10-K, 10-Q, 10-K/A, ...
  filed_at     TEXT NOT NULL,           -- ISO date
  period_of_report TEXT,                -- ISO date
  is_amendment INTEGER NOT NULL DEFAULT 0,
  amends       TEXT,                   -- accession_no this amends
  source_url   TEXT,
  raw_path     TEXT                    -- local file path if cached
);
CREATE INDEX idx_filing_cik_filed ON filing(cik, filed_at DESC);

-- Filings - extended with the Item 4.02 marker.
-- (filing table above adds these columns; shown together for clarity)
ALTER TABLE filing ADD COLUMN
  item_4_02_8k INTEGER NOT NULL DEFAULT 0; -- 1 iff form_type='8-K' AND items contains '4.02'

-- Periods (canonicalized fiscal periods).
-- fiscal_quarter uses 0 for annual rows (NOT NULL) so the UNIQUE
-- constraint actually constrains annual periods. kind is derived
-- from fiscal_quarter via CHECK and stored for query convenience.
CREATE TABLE period (
  id          INTEGER PRIMARY KEY,
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  fiscal_year  INTEGER NOT NULL,
  fiscal_quarter INTEGER NOT NULL,      -- 0 = annual, 1..4 = quarterly
  fiscal_year_end TEXT NOT NULL,        -- MMDD; lets us detect FYE changes
  start_date   TEXT NOT NULL,           -- ISO date
  end_date     TEXT NOT NULL,           -- ISO date
  kind         TEXT NOT NULL,           -- 'annual' | 'quarterly'
  is_53_week   INTEGER NOT NULL DEFAULT 0, -- 1 for 53-week fiscal years (retailers)
  CHECK (fiscal_quarter BETWEEN 0 AND 4),
  CHECK ((fiscal_quarter = 0 AND kind = 'annual') OR
         (fiscal_quarter BETWEEN 1 AND 4 AND kind = 'quarterly')),
  UNIQUE (cik, fiscal_year, fiscal_quarter)
);
CREATE INDEX idx_period_cik_year ON period(cik, fiscal_year);

-- Raw facts (1:1 with what we got from EDGAR / XBRL).
-- value_numeric is in absolute base unit. The companyfacts JSON path
-- always returns absolute values; the XBRL XML fallback applies its
-- own scaling at parse time before insert. There is no scale column.
CREATE TABLE raw_fact (
  id          INTEGER PRIMARY KEY,
```

```

cik            TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
accession_no   TEXT NOT NULL REFERENCES filing(accession_no) ON DELETE RESTRICT,
taxonomy       TEXT NOT NULL,                -- 'us-gaap', 'ifrs-full', 'dei', ...
concept        TEXT NOT NULL,                -- e.g. 'Revenues'
unit           TEXT NOT NULL,                -- 'USD', 'shares', 'USD/shares'
value_numeric  REAL NOT NULL,               -- absolute value in declared unit
period_start   TEXT,                        -- NULL iff is_instant=1
period_end     TEXT NOT NULL,               -- end date for instants
is_instant     INTEGER NOT NULL,
fy             INTEGER,
fp             TEXT,                        -- 'FY', 'Q1', 'Q2', 'Q3', 'Q4'
source_kind    TEXT NOT NULL,               -- 'xbrl_api' | 'xbrl_xml'
ingested_at    TEXT NOT NULL,
-- Natural-key UNIQUE so refresh re-ingestion is idempotent without
-- creating duplicate raw_fact rows. Matches what `companyfacts`
-- guarantees uniqueness over per accession.
UNIQUE (cik, accession_no, taxonomy, concept, unit, period_start, period_end, fp)
);
CREATE INDEX idx_raw_cik_concept ON raw_fact(cik, taxonomy, concept);
CREATE INDEX idx_raw_filing ON raw_fact(accession_no);

-- Canonical / normalized facts.
-- Multiple alternates may exist per (cik, metric, period_id); exactly
-- one carries is_primary=1. The dashboard reads only primary, current
-- (not superseded) rows.
CREATE TABLE normalized_fact (
  id            INTEGER PRIMARY KEY,
  cik           TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  metric        TEXT NOT NULL,              -- canonical metric name
  period_id     INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,
  value         REAL NOT NULL,              -- in base unit (USD, shares, ...)
  unit          TEXT NOT NULL,
  source_fact_id INTEGER NOT NULL REFERENCES raw_fact(id) ON DELETE RESTRICT,
  source_kind    TEXT NOT NULL,             -- 'xbrl_api' | 'xbrl_xml'
  is_primary     INTEGER NOT NULL DEFAULT 1, -- 1 = canonical chosen value, 0 = alternate
  -- superseded_by is a linked list: each prior value points at its
  -- IMMEDIATE successor (not a flat pointer to the latest). Walking
  -- the chain reconstructs the full restatement history. Cycles are
  -- prevented by the trigger declared below.
  superseded_by INTEGER REFERENCES normalized_fact(id) ON DELETE RESTRICT,
  ingested_at    TEXT NOT NULL,
  UNIQUE (cik, metric, period_id, source_fact_id)
);
-- Exactly one primary, non-superseded row per (cik, metric, period_id):
CREATE UNIQUE INDEX idx_norm_primary_current
  ON normalized_fact (cik, metric, period_id)
  WHERE is_primary = 1 AND superseded_by IS NULL;
CREATE INDEX idx_norm_cik_metric_period ON normalized_fact(cik, metric, period_id);

-- Cycle protection on supersession chain.
CREATE TRIGGER trg_norm_no_cycle
BEFORE UPDATE OF superseded_by ON normalized_fact
WHEN NEW.superseded_by IS NOT NULL AND EXISTS (
  WITH RECURSIVE chain(id) AS (
    SELECT NEW.superseded_by
    UNION ALL
    SELECT nf.superseded_by
    FROM normalized_fact nf JOIN chain ON nf.id = chain.id
    WHERE nf.superseded_by IS NOT NULL
  )
  SELECT 1 FROM chain WHERE id = NEW.id
)

```

```

BEGIN
  SELECT RAISE(ABORT, 'normalized_fact.superseded_by would create a cycle');
END;

-- 8-K Item 4.02 restatement announcements: which periods are flagged
-- unreliable by which 8-K filing. Cleared automatically by the
-- "is unresolved" query in §8.7 once amendments cover the periods.
CREATE TABLE restatement_announcement (
  id          INTEGER PRIMARY KEY,
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  accession_no TEXT NOT NULL REFERENCES filing(accession_no) ON DELETE RESTRICT,
  affected_period_id INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,
  filed_at    TEXT NOT NULL,
  ingested_at TEXT NOT NULL,
  UNIQUE (accession_no, affected_period_id)
);
CREATE INDEX idx_restate_cik ON restatement_announcement(cik);

-- Historical EOD prices (one row per ticker per date). Sized for
-- ~10K trading days over 40 years × N tickers – small.
CREATE TABLE historical_price (
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  date        TEXT NOT NULL,           -- ISO date
  close       REAL NOT NULL,
  source      TEXT NOT NULL,           -- adapter id
  ingested_at TEXT NOT NULL,
  PRIMARY KEY (cik, date)
);

-- Derived metrics (computed from normalized facts and/or prices).
CREATE TABLE derived_metric (
  id          INTEGER PRIMARY KEY,
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  formula_id  TEXT NOT NULL,           -- 'fcf_v1', 'total_debt_v1', 'historical_market_cap_v1',
  period_id   INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,
  value       REAL,
  is_complete INTEGER NOT NULL,        -- 0 if any input missing
  computed_at TEXT NOT NULL,
  UNIQUE (cik, formula_id, period_id)
);

-- Ingestion diagnostics (observable to the user).
CREATE TABLE ingestion_event (
  id          INTEGER PRIMARY KEY,
  cik         TEXT REFERENCES company(cik) ON DELETE RESTRICT,
  accession_no TEXT REFERENCES filing(accession_no) ON DELETE RESTRICT,
  stage       TEXT NOT NULL,           -- discover|download|parse|normalize|persist|item_4_02|price
  level       TEXT NOT NULL,           -- info|warn|error
  user_visible INTEGER NOT NULL DEFAULT 0, -- 1 if also surfaced on the dashboard, not only Diagnostic
  message     TEXT NOT NULL,
  detail_json TEXT,
  occurred_at TEXT NOT NULL
);
CREATE INDEX idx_ing_cik_time ON ingestion_event(cik, occurred_at DESC);

```

Notes:

- `raw_fact` and `normalized_fact` are kept as **separate tables** rather than overwriting raw data. This is what makes traceability cheap: a normalized row points at exactly one raw row, which points at exactly one filing.
- `superseded_by` is a **linked list**, not a flat pointer. When amendment A2 supersedes A1 (which already superseded the original O), the chain is `0 → A1 → A2`. The current value is whichever row has `superseded_by IS NULL`. The lineage

panel walks the chain backward to show the full restatement history.

- Cycle protection on `superseded_by` is enforced by the SQLite trigger `trg_norm_no_cycle` declared above.
- All child tables have explicit `cik` FKs. The default `ON DELETE RESTRICT` policy enforces FR-004 (removal preserves cached data unless explicitly cleared) — actually deleting a company will fail until an explicit "uncache" operation drops dependent rows.
- Confidence scoring is deferred to V2. The V1 alternates model uses a binary `is_primary` flag instead; the lineage panel surfaces alternate facts on demand and Diagnostics tracks the resolution decision.
- The schema is migration-managed; every change ships as a numbered SQL file, not a hand edit.

6.4 Filesystem layout

```
~/Library/Application Support/com.<vendor>.econ-project/
├── data.sqlite                # canonical DB
├── data.sqlite-wal           # WAL (journal_mode=WAL)
├── filings/
│   └── <cik>/
│       ├── companyfacts.json # SEC's pre-extracted XBRL facts, one per company
│       ├── submissions.json  # SEC filing index, one per company
│       ├── prices.json       # cached historical EOD price series
│       └── <accession_no>/    # per-filing artifacts (sparse)
│           ├── primary_doc.xml # raw XBRL instance document (XBRL-XML fallback only)
│           ├── item_4_02.html  # 8-K Item 4.02 primary document (Item 4.02 8-Ks only)
│           └── metadata.json
├── logs/
│   └── econ-project.<date>.log
└── config.json               # user prefs
```

The three top-level files under each `<cik>/` directory (`companyfacts.json`, `submissions.json`, `prices.json`) are the per-company artifacts V1 fetches in the normal path. The per-`<accession_no>/` subdirectory is created only for filings that require per-filing artifacts: an Item 4.02 8-K (HTML document needed for period extraction) or a `companyfacts` gap that triggered the XBRL-XML fallback. For most companies, that subdirectory is empty or absent.

Raw payloads are kept on disk (not in SQLite blobs) so they can be inspected, backed up, or re-processed without touching the database. The `raw_path` column in `filing` points to the relevant directory or file.

7. SEC Integration

7.1 Endpoints used

Purpose	Endpoint	Notes
Ticker → CIK lookup	<code>https://www.sec.gov/files/company_tickers.json</code>	Cached locally; refresh weekly
Filing index	<code>https://data.sec.gov/submissions/CIK{cik10}.json</code>	Per-company list of filings (form type, accession, dates). Used to know what filings exist; does not contain financial values.
All facts (primary)	<code>https://data.sec.gov/api/xbrl/companyfacts/CIK{cik10}.json</code>	SEC's pre-extracted XBRL facts , aggregated across all of a company's 10-K / 10-Q / 8-K filings. Not the filings themselves. One JSON per company. This is V1's main data source.
Single concept (fallback)	<code>https://data.sec.gov/api/xbrl/companyconcept/CIK{cik10}/{taxonomy}/{tag}.json</code>	Same data sliced by concept. Used for targeted re-fetch when <code>companyfacts</code> has a gap for a known concept.
Raw filing index	<code>https://www.sec.gov/cgi-bin/browse-edgar?...</code>	Used only to locate the path of a specific filing's XBRL instance document when the fallback path is required.
Raw XBRL document	<code>https://www.sec.gov/Archives/edgar/data/{cik}/{accession-stripped}/{primary_doc}</code>	The XBRL instance document (XML) inside a specific filing.

Purpose	Endpoint	Notes
		Used by the XBRL-XML fallback path when <code>companyfacts</code> has a gap.
8-K Item 4.02 primary document	<code>https://www.sec.gov/Archives/edgar/data/{cik}/{accession-stripped}/{primary_doc}</code>	The 8-K's primary document (HTML / iXBRL). Downloaded only when an Item 4.02 8-K is in the submissions index. Parsed narrowly to extract the affected fiscal periods. No financial values are extracted; no other 8-K item types are parsed.

7.2 Compliance

- **User-Agent:** the SEC requires a descriptive `User-Agent` of the form `AppName/Version contact@example.com`. Source: `https://www.sec.gov/os/accessing-edgar-data` (verify against current text before release; SEC has previously rejected requests with missing or vague User-Agent strings). The string is configurable in `config.json` and surfaced in onboarding so the user understands what is being sent.
- **Rate limit:** SEC publishes a ceiling of 10 requests/second per client. Source: `https://www.sec.gov/os/accessing-edgar-data` (re-verify before release). The client enforces a token-bucket limiter at 5 req/s by default with jittered backoff on 429/5xx; the limiter is shared across all sources (EDGAR + 8-K HTML + `companyconcept`) so concurrent ingestion jobs cannot stack into a violation.
- **No mass crawling:** the app fetches per-company on user action, never speculatively.

7.3 Caching and refresh

- `company_tickers.json` cached for 7 days.
- `submissions/CIK{cik}.json` cached for 24 hours, with `If-Modified-Since` revalidation.
- `companyfacts/CIK{cik}.json` re-fetched on every user-initiated refresh, with `ETag / Last-Modified` revalidation. The artifact is one document containing every fact ever filed for the company; SEC re-processing can change facts under a stable accession number, so the refresh path always re-derives `raw_fact` rows for the file's contents and relies on the natural-key UNIQUE constraint to no-op duplicates.
- Item 4.02 8-K HTML and per-filing XBRL XML documents are immutable once fetched — accession numbers never get reused.
- Historical EOD prices: see §7.5.

7.4 Failure modes

Condition	Response
No network	UI surfaces "offline"; ingestion buttons disabled; navigation of cached data unaffected; current-market-cap widget shows "unavailable" without affecting historical chart
429 rate limit	Exponential backoff up to 60s, then surface a user-visible warning
5xx	Same as 429 with a different message
Schema change in EDGAR JSON	Fail closed for that fact; record an <code>ingestion_event</code> with the offending payload; do not overwrite previously-good data
Missing concept for a known metric	Mark the period as having a gap; the UI shows an explicit " — "
Item 4.02 period-extraction unable to identify affected periods	Block ingestion of the 4.02 with a high-severity, user-visible <code>ingestion_event</code> . The system does not fall back to over-flagging; an unparsed 4.02 is treated as a defect to fix in the parser, not a runtime degradation.
Market-data adapter unavailable at ingestion	Persist all other ingested data; record an <code>ingestion_event</code> ; the historical-market-cap point for the affected filing is filled in on the next refresh.
Market-data adapter unavailable at runtime	Hide the current-market-cap widget; rest of dashboard unaffected.

7.5 Market-data integration

Historical and current price data is consumed through a single `MarketDataAdapter` trait so the choice of provider is replaceable. V1 commits to **Yahoo Finance** as the default adapter (no API key required, public historical-quote endpoint), behind a small abstraction so a paid provider (IEX Cloud, Polygon, Alpha Vantage) can be substituted without changing call sites.

- **Historical EOD prices** are fetched per ticker once at first ingestion and refreshed incrementally on each user-initiated refresh (only the new dates since `MAX(date)` in `historical_price` for that CIK). Cached to `<cik>/prices.json` and persisted to the `historical_price` table.
- **Current price** is fetched on demand when the dashboard's current-market-cap widget mounts and the device is online. No caching; an unavailable response is surfaced gracefully.
- The adapter is host-allowlisted in the `request` client (see §15) — it is the only non-SEC outbound destination V1 reaches.

8. Normalization Engine

The PRD elevates normalization to a *core architectural requirement*. The engine is implemented as a pure-functions module with the following responsibilities:

8.1 Concept mapping

A concept map associates external XBRL concepts with canonical metrics:

```
// Sketch
ConceptMap {
  revenue: [
    ("us-gaap", "Revenues"),
    ("us-gaap", "RevenueFromContractWithCustomerExcludingAssessedTax"),
    ("us-gaap", "SalesRevenueNet"),
    ("us-gaap", "SalesRevenueGoodsNet"),
    ("us-gaap", "SalesRevenueServicesNet"),
  ],
  capital_expenditures: [
    ("us-gaap", "PaymentsToAcquirePropertyPlantAndEquipment"),
    ("us-gaap", "PaymentsToAcquireProductiveAssets"),
  ],
  // ...
}
```

When multiple candidate facts could populate the same `(metric, period)`, a **resolution rule** selects exactly one as primary (`is_primary = 1`) and records the others as alternates (`is_primary = 0`). The rule order matches PRD §11.3:

1. **Prefer the most recent amendment in the supersession chain.** If a 10-K/A or 10-Q/A covers the period, its values are primary; the original 10-K / 10-Q values are kept as alternates and the amendment supersedes them via `superseded_by` (see §8.5).
2. **Prefer the original filing only when no amendment covers the period.**
3. **Prefer the canonical primary concept** within the chosen filing's facts. If that concept is absent, fall through the catalog's ordered fallback list (see the §6.2 catalog and the `total_debt` definition for an example).
4. **Prefer `xbrl_api` (companyfacts) over `xbrl_xml`** (raw XBRL fallback) when the same value appears in both — the API is the SEC's canonicalized read-side.

The decision, the inputs that contributed, and any rejected alternates are written to `ingestion_event` so the user can audit the resolution from the Diagnostics tab.

8.2 Period reconciliation

The most error-prone area. The engine:

- **Aligns fiscal periods to the company's `fiscal_year_end`.** A company with FYE 09-30 (like Apple) has its FY2024 covering Oct 2023 – Sep 2024. Each `period` row carries the FYE in effect at the time so historic alignments are preserved if the FYE later changes.
- **Distinguishes instant from duration facts.** Balance-sheet items are stored at a point in time (`is_instant = 1`); income- and cash-flow items are stored over a duration. Mismatches are diagnostic-flagged and not silently coerced.
- **Handles YTD vs. quarterly reporting.** Many filings report Q3 as the 9-month YTD total. The engine *always* derives the single-quarter value when only YTD is reported ($Q3 = 9M\ YTD - H1\ YTD$; $Q2 = H1\ YTD - Q1$; etc.), preferring the directly reported quarter when it is available. Every derivation writes a lineage record naming the YTD inputs used.
- **Handles 52/53-week fiscal calendars** (common for retailers like Costco, Apple-pre-2008, Cisco). The engine detects 53-week years via `end_date - start_date > 364 days` and sets `period.is_53_week = 1`. Year-over-year comparisons on 53-week years are explicitly flagged in the UI rather than silently averaged with 52-week years.
- **Handles fiscal-year-end changes.** When a filing's `period_of_report` indicates a different FYE than the company's previously recorded FYE, the engine inserts a high-severity, user-visible `ingestion_event` and creates new `period` rows under the new FYE. It does not retroactively rewrite historical periods. The UI surfaces a banner on the Company Dashboard for any company whose FYE has changed at any point in its history.
- **Refuses to invent periods.** If a fact's period boundaries cannot be reconciled to a `period` row (e.g., a stub period after IPO, an unusual transition period after FYE change), the fact is parked in `raw_fact` but not promoted to `normalized_fact`, and a user-visible `ingestion_event` is written. Accuracy is not traded for coverage.

8.3 Unit normalization

Each `raw_fact` records its declared unit. The `companyfacts` JSON path returns absolute values; the XBRL XML fallback applies `decimals / unitRef` scaling at parse time before insert. Normalization then converts to base units:

- Currency facts → base currency (USD).
- Share counts → absolute share counts.
- Per-share ratios → kept as ratios with explicit `USD/shares` unit.
- Non-USD reporting currencies (foreign private issuers): the engine consults the company's `dei:EntityReportingCurrencyISOCode` fact and applies an FX conversion to USD using the daily ECB / Fed reference rate for `period_end`. The original-currency value, the rate used, and its source are all recorded in lineage; both the original-currency and USD values are queryable. The dashboard always renders USD by default with an "as reported in " tooltip; the user can toggle. This satisfies PRD §6.5's currency-inconsistency requirement without dropping non-USD filers.

8.4 Sign normalization

Cash-flow concepts in particular use mixed sign conventions across companies (CapEx may be reported as positive payments or negative cash flow). The catalog defines the storage sign convention; the normalizer applies it deterministically and records both the original sign and the applied transform in lineage.

8.5 Restatement handling

When a 10-K/A or 10-Q/A is ingested:

1. New `filing` row marked `is_amendment = 1`, `amends = <original accession>`.
2. New `raw_fact` rows inserted (idempotent via the natural-key UNIQUE).
3. New `normalized_fact` rows inserted with `is_primary = 1`. The supersession chain is updated as a **linked list**: the immediate predecessor (the previously-current row with `superseded_by` IS NULL for the same (`cik`, `metric`, `period_id`) and `is_primary = 1`) gets its `superseded_by` set to the new row's id. Earlier rows in the chain are not modified; their `superseded_by` already points forward.
4. The trigger `trg_norm_no_cycle` (see §6.3) refuses cycles.
5. Read queries default to `WHERE is_primary = 1 AND superseded_by IS NULL`. The lineage panel walks the chain backward (`superseded_by` is per-row → each row also exposes "what supersedes me" via the inverse query) to show the full restatement history with each step's filing accession and date.

Multi-step amendments (10-K → 10-K/A → 10-K/A2): each amendment supersedes only its immediate predecessor. The chain reads as `0 → A1 → A2`; `WHERE superseded_by IS NULL` returns A2 alone, which is the current view; the lineage panel walks `0 ← A1 ← A2` on demand. Cycle protection prevents pathological inputs from corrupting the chain.

8.6 Conflict surfacing

The PRD says: *"The application must never silently discard normalization conflicts or ambiguities."*

Every place the engine has to make a choice writes an `ingestion_event`. The `level` column distinguishes `info` (routine derivations like YTD→Q3), `warn` (resolved conflicts where alternates were demoted), and `error` (unresolved cases where the fact was not promoted). The `user_visible` column flags the events that surface on the dashboard itself rather than only in the Diagnostics tab — `error` events and FYE-change banners are user-visible by default; routine `info` events stay in Diagnostics.

8.7 Item 4.02 restatement-warning handling

When an Item 4.02 8-K is identified in the submissions index (`filing.item_4_02_8k = 1`):

1. The 8-K's primary document is downloaded and saved to `<accession_no>/item_4_02.html`.
2. A dedicated parser extracts the specific fiscal periods the disclosure flags as unreliable. The parser must do this with full accuracy regardless of phrasing or structured-tag presence; an inability to identify the affected periods is a parser bug, not a runtime case to handle gracefully (see §7.4).
3. For each identified period, a `restatement_announcement` row is inserted referencing the 4.02 accession and the affected `period_id`.
4. The dashboard renders a per-period warning whenever the resolution query returns at least one row:


```
sql SELECT 1 FROM restatement_announcement ra WHERE ra.cik = ? AND NOT EXISTS ( SELECT 1 FROM filing amend WHERE amend.cik = ra.cik AND amend.is_amendment = 1 AND amend.filed_at > ra.filed_at AND amend.period_of_report = ( SELECT period.end_date FROM period WHERE period.id = ra.affected_period_id ) );
```

The warning clears period-by-period as covering amendments are ingested.
5. **No financial values are extracted from the 8-K itself.** Restated values land via the normal `raw_fact` / `normalized_fact` / `superseded_by` path when the 10-K/A or 10-Q/A is filed and ingested.

9. Ingestion Pipeline

9.1 Stages

Discover → Download → Parse → Normalize → Persist

Each stage takes a typed input and produces a typed output; each is independently testable with golden fixtures.

Stage	Input	Output	Side effects
Discover	Ticker	CIK + filing index from <code>submissions.json</code> , with Item 4.02 8-Ks flagged	None
Download (facts)	CIK	<code>companyfacts.json</code> (primary); per-accession XBRL instance XML (fallback only)	Filesystem
Download (4.02)	Item 4.02 8-K accessions	8-K primary documents (HTML / iXBRL)	Filesystem
Download (prices)	CIK + new date range	Historical EOD prices since last refresh	Filesystem + DB
Parse	<code>companyfacts.json</code> , fallback XBRL XML, 8-K Item 4.02 documents	<code>RawFact</code> records + <code>RestatementAnnouncement</code> records	None
Normalize	<code>RawFacts</code> + <code>ConceptMap</code>	<code>NormalizedFacts</code> + <code>Diagnostics</code>	None
Persist	<code>NormalizedFacts</code> + <code>RawFacts</code> + <code>RestatementAnnouncements</code> + <code>Prices</code> + <code>Diagnostics</code>	Persisted state	DB

Normal-path traffic. For a typical ingestion of a company with no Item 4.02 history, V1 makes three HTTP requests: one to `data.sec.gov/submissions/CIK*.json`, one to `data.sec.gov/api/xbrl/companyfacts/CIK*.json`, and one to the market-data adapter for historical prices. When Item 4.02 8-Ks exist in the submissions index, one additional request per 4.02 fetches its primary document. Per-filing XBRL XML requests are made *only* on the fallback path. 10-K / 10-Q HTML / iXBRL documents are never fetched.

9.2 Concurrency model

- Stages run as `tokio` tasks within a per-ingestion job.
- Network I/O (Discover, Download) is rate-limited globally via a shared token bucket so multiple ingestion jobs cannot exceed SEC rate limits.
- DB writes happen on a single writer task that owns the SQLite write connection; readers use a separate pool. This avoids `SQLITE_BUSY` while WAL is in use.
- Progress is published over a Tauri event channel (`ingestion://progress/<job_id>`) for the UI to render progress bars.

9.3 Idempotency

- Ingestion is keyed by `accession_no` . Re-running ingestion for an already-persisted filing is a no-op.
- A partially-completed job can be resumed: the persist stage records a checkpoint per filing, and Discover skips filings already at the latest stage.

9.4 Failure semantics

Per the PRD, partial ingestion is acceptable as long as it is visible. The pipeline:

- **Continues** when a single filing fails to parse — other filings still produce data.
 - **Halts** the job when the database itself is unhealthy (disk full, corruption signal).
 - **Records every failure** as an `ingestion_event` row so the user can see what was skipped.
-

10. Derived Metric Engine

V1 ships a small fixed catalog of formulas. Each formula is a Rust function that declares the canonical metrics (and, where relevant, the historical-price inputs) it consumes, and returns either a value with a lineage record or `is_complete = 0` with the missing-input list.

V1's registered formulas:

Formula id	Output metric	Inputs	Notes
fcf_v1	(derived per period)	net_income , depreciation_amortization , capital_expenditures	FCF = NI + D&A – CapEx (CapEx is sign-normalized positive at storage)
total_debt_v1	total_debt	long_term_debt , current_debt	Sum, with documented per-input fallback chain (see §6.2)
gross_profit_v1	gross_profit	revenue , cost_of_revenue	Used only when gross_profit is not directly tagged in companyfacts
historical_market_cap_v1	historical_market_cap	historical_price[filed_at] , shares_outstanding_basic	Computed once per filing at ingestion; persisted to derived_metric ; offline-available
current_market_cap_v1	current_market_cap	live price, shares_outstanding_basic (latest)	Computed on demand; not persisted; widget hidden when offline

If any input is missing, the result is `is_complete = 0` with no value, and the UI displays a gap rather than a fabricated number. This satisfies FR-030 / FR-031 / FR-032 and the PRD principle of "explicit gaps, no fabricated estimates."

The engine is built so V2's user-defined formulas can plug in via a registry — but V1 only registers compile-time functions.

11. UI Architecture

11.1 Routes

Route	Purpose
/	Home: saved companies, add ticker, refresh status
/c/:ticker	Company dashboard
/c/:ticker/statement/:kind	Income / Balance / Cash flow tables
/c/:ticker/metric/:metric	Drill-down: per-metric history + lineage
/c/:ticker/diagnostics	Ingestion diagnostics & data-quality view

11.2 Component composition

- **Dashboard** = `<RestatementBanner/>` + `<FyeChangeBanner/>` + `<SummaryWidgets/>` + `<ChartGrid/>` + `<StatementsTable/>`.
- **RestatementBanner** renders whenever §8.7's resolution query returns at least one period for the current company. Names the affected period(s) and the date the 4.02 was filed. Clears period-by-period as covering amendments are ingested.
- **FyeChangeBanner** renders for any company whose fiscal-year-end has changed at any point in its ingested history.

- **SummaryWidgets** displays revenue, net income, cash, total debt, FCF, and historical market cap as the latest reported annual value with a sparkline of recent quarters. A `<CurrentMarketCap/>` sub-widget is rendered when the live price source is online; when offline or the source is down, it shows an explicit "current market cap unavailable" state without affecting the rest of the dashboard.
- **ChartGrid** exposes Annual / Quarterly toggle (FR-051), default 10y, configurable to 20y. Charts use ECharts via `echarts-for-react` with chart-instance reuse (`notMerge: false`, `lazyUpdate: true`) and `useMemo` on series props so re-renders do not stall.
- **StatementsTable** is a virtualized table (TanStack Table) with row drill-down → `/c/:ticker/metric/:metric`. Rows for periods flagged by an unresolved Item 4.02 carry a per-row "unreliable" indicator.
- **Lineage panel** is a side drawer that shows: filing accession, form type, filing date, XBRL concept, raw value, sign transform, supersession chain, and any FX conversion applied for non-USD filers (FR-060 / FR-061 / FR-062).

11.3 State and data flow

- All data is fetched via Tauri IPC commands wrapped as TanStack Query hooks.
- The query cache is the de-facto in-memory state; there is no Redux.
- Mutations (add company, refresh) invalidate relevant queries and trigger re-fetch.
- Long-running jobs (ingestion) emit progress events; the UI subscribes via Tauri event listeners.

11.4 Design system

- Density: ~13 px base font; tabular numerals for all financial values.
- Color: a constrained palette (neutrals + one accent) with explicit positive/negative colors for variances. No gradients, no shadows beyond elevation-1.
- Typography: a single sans-serif (system stack: SF Pro on macOS) plus a mono stack for raw values in the lineage panel.
- Loading states: skeleton placeholders for tables; explicit "missing data" markers (PRD §10.5 error visibility).

12. Concurrency, Performance, and Caching

12.1 SQLite tuning

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA temp_store = MEMORY;
PRAGMA mmap_size = 268435456; -- 256 MB
PRAGMA foreign_keys = ON;
```

WAL is essential so the UI can read while ingestion writes.

12.2 Connection model

- One **write** connection, owned by the persist worker.
- A **read pool** of 4 connections shared across IPC handlers.

12.3 Query patterns

- The dashboard's hot path is `SELECT metric, period_id, value FROM normalized_fact WHERE cik = ? AND is_primary = 1 AND superseded_by IS NULL ORDER BY period_id` — covered by the partial unique index

`idx_norm_primary_current` .

- All frequent reads are bounded by `cik` ; an index on `(cik, ...)` covers them.
- The lineage panel's "what supersedes me" walk is the inverse: `SELECT id FROM normalized_fact WHERE superseded_by = ?` — covered by an additional non-unique index on `superseded_by` (added in the same migration as the table).

12.4 Caching

- Filing metadata: in DB, `last_refreshed` per company.
 - Derived metrics: persisted in `derived_metric` ; recomputed only when an input changes.
 - UI: TanStack Query with a 5-minute stale time for IPC results inside a session.
-

13. Observability

13.1 Logging

- `tracing` with a JSON layer to a rotating file under `~/Library/.../logs/` .
- A console layer when launched with `RUST_LOG=debug` .
- Log levels: `info` for stage transitions, `warn` for fallbacks and conflicts, `error` for skipped data.

13.2 In-app diagnostics

Every interesting decision the system makes during ingestion writes a row to `ingestion_event` . The **Diagnostics** tab in the company dashboard renders this table with filters. This is what makes the PRD's "transparency" requirement concrete.

13.3 No telemetry

Nothing is sent off the device. The app does not ping a vendor URL, does not check for updates, and does not phone home. Outbound traffic is gated by **two independent mechanisms**:

1. **Tauri 2 capabilities** (declared in `src-tauri/capabilities/*.json`) restrict which `#[tauri::command]` handlers the `WebView` can invoke. The frontend has no direct HTTP capability; it can only request fetches via specific commands.
2. **Rust-side host allowlist** in the `request` client builder restricts which hostnames the Rust core can call out to: `www.sec.gov` , `data.sec.gov` , and the configured market-data adapter's host. Any other URL fails closed at the HTTP layer regardless of code path. (Tauri's permission system does not govern Rust-side `request` calls — the host allowlist is what actually enforces this.)

Both layers are required; neither is sufficient on its own.

14. Error Handling

14.1 Layered errors

- **Source layer:** `SourceError` with variants for HTTP status, parse failures, schema violations.
- **Pipeline layer:** `IngestionError` wraps `SourceError` and adds stage context.
- **Service layer:** `AppError` is the top-level enum exposed across IPC; it carries a stable `code` string for the UI to switch on, plus a localized message.

14.2 IPC error contract

Every Tauri command returns `Result<T, AppError>`. The frontend's typed client maps `AppError.code` to user-visible messaging; the underlying message and detail are surfaced in a "Show details" affordance for advanced users.

14.3 Partial success

A common case: ingestion succeeds for 18 of 20 filings. The IPC response is `Ok(IngestionSummary { succeeded, skipped: [...] })` and the UI shows a non-blocking notice with a link to Diagnostics.

15. Security and Privacy

Concern	Mitigation
Outbound traffic	Tauri 2 capabilities (gates JS→Rust command invocation) + <code>request host allowlist</code> (gates Rust→Internet) — both required, see §13.3
Telemetry	None — zero analytics, zero remote config
Local data integrity	SQLite WAL + <code>synchronous = NORMAL ; PRAGMA integrity_check</code> on startup
Filesystem permissions	Data stored under <code>~/Library/Application Support/<bundle>/</code> ; App Sandbox enabled with <code>com.apple.security.network.client</code> for outbound HTTP
Code signing	Developer ID-signed and notarized for distribution; required before any external build
Dependencies	Minimal set; <code>cargo audit</code> in CI; <code>pnpm audit</code> for the React side
Secrets	None stored; SEC has no auth; the User-Agent contact email is user-supplied

16. Build, Packaging, and Distribution

- **Local development:** `pnpm tauri dev`.
- **Production build:** `pnpm tauri build` produces `.dmg` and `.app`.
- **Code signing & notarization:** Developer ID-signed and notarized via Apple's notarytool. App Sandbox enabled. Required before any external build is distributed.
- **Distribution channel (V1):** developer-distributed `.dmg` from a personal-website download. **No auto-update mechanism in V1.** App Store distribution and managed-update channels are V2+.
- **Unit tests:** `cargo test` for Rust, `vitest` for the React side.
- **Integration tests:** golden-fixture tests that ingest a date-pinned, checked-in copy of an EDGAR `companyfacts` JSON for sample companies (Apple as the canonical fixture; one bank/insurer/REIT/foreign-private-issuer as edge-case fixtures). Fixture-pinning date is recorded next to each fixture so future regressions are obvious. Item 4.02 parser tests use a corpus of past Item 4.02 8-Ks across multiple phrasings.
- **CI (GitHub Actions, post-V1):** macOS runner, lint + test + bundle.

17. Future Extensibility

The architecture has explicit seams for the V2/V3 directions in the PRD:

Future feature	Seam
Multi-company comparison	Repository queries are already keyed by <code>cik</code> ; UI introduces a new layer that joins multiple companies. No schema change.
User-defined formulas	The derived-metric engine already accepts a registry; V2 adds a parser + sandboxed evaluator.
Peer benchmarking	A <code>peer_group</code> table joins companies; the metric service gains a <code>for_peers</code> API.
Export	A new <code>export/</code> module reads from <code>normalized_fact</code> and <code>derived_metric</code> and writes CSV/Excel/PDF. The data is unchanged.
Plugin SDK	The stable Rust traits (<code>MetricProvider</code> , <code>SourceAdapter</code> , <code>MarketDataAdapter</code>) are already what the in-tree modules implement. The eventual loading mechanism (dynamic library, <code>WebAssembly</code> , or in-process Rust extension shipped as part of a plugin bundle) is a V3 design choice with non-trivial security and code-signing implications and is not decided in V1.
Local LLM	The lineage and normalization layers already produce structured records that can be fed as context.

The boundary that matters most is keeping `raw_fact` separate from `normalized_fact` . Every future feature will benefit from the audit trail this provides.

18. Risks and Open Questions

Risk	Likelihood	Mitigation
EDGAR companyfacts API changes its schema	Low	Fail closed per fact; raw XBRL fallback; integration tests on golden fixtures
Concept-map coverage is incomplete for some industries (banks, insurers, REITs, foreign private issuers)	High	V1 ships a vetted catalog for non-financials, banks, insurers, REITs, and FPIs; per-industry golden fixtures gate releases
Item 4.02 parser fails to identify affected periods on a novel phrasing	Medium	Block ingestion for the offending 4.02 with a high-severity user-visible event; add the phrasing to the test corpus and ship a parser fix. No over-flagging fallback.
Companies with non-USD reporting currency	Medium	Ingest fully; store both original-currency and USD-converted values; record FX rate and source in lineage
Restatement chains spanning >1 amendment	Medium	Linked-list <code>superseded_by</code> chain; cycle protection trigger; lineage panel walks the chain
User runs ingestion for many companies in parallel and hits SEC rate limits	Medium	Global token bucket shared across all sources; queue rather than reject
SQLite corruption from disk-full	Low	WAL + <code>PRAGMA integrity_check</code> on startup; APFS copy-on-write helps
Ticker→CIK lookup ambiguity (multiple share classes)	Medium	Disambiguation UI when the lookup returns >1 candidate
Market-data adapter (Yahoo Finance) becomes unreliable	Medium	<code>MarketDataAdapter</code> trait keeps the call sites unchanged; swap provider via configuration; degraded UI when adapter is unavailable

Open questions

The architecture's previous open-questions list has been resolved by the V1 Q&A:

1. `~~Reporting currency support~~` → V1 ingests non-USD filers fully (§8.3).
2. `~~Quarterly derivation policy~~` → Always derive single-quarter values from YTD when a directly reported quarter is unavailable (§8.2).
3. `~~Market cap source~~` → Yahoo Finance via the `MarketDataAdapter` trait; historical EOD persisted, current value live-only (§7.5, §10).
4. `~~Code signing timing~~` → Required before any external build (§16).

No new open questions remain at the architecture-doc altitude. Implementation-level choices are tracked in code reviews and ADRs.

19. Appendices

A. Sample SEC URLs (for reference)

- Apple ticker→CIK: <https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany&CIK=AAPL&type=10-K&dateb=&owner=include&count=40>
- Apple submissions: <https://data.sec.gov/submissions/CIK0000320193.json>
- Apple company facts: <https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json>
- Apple revenues concept: <https://data.sec.gov/api/xbrl/companyconcept/CIK0000320193/us-gaap/Revenues.json>

B. Glossary

- **CIK** — Central Index Key, SEC's unique 10-digit company identifier.
- **XBRL** — eXtensible Business Reporting Language; the XML-based standard used in SEC structured filings.
- **Taxonomy** — a published vocabulary of XBRL concepts (e.g., `us-gaap`, `ifrs-full`).
- **Concept** — a single tagged datapoint within a taxonomy (e.g., `us-gaap:Revenues`).
- **Accession number** — SEC's unique identifier for a single filing submission.
- **Form 10-K / 10-Q** — annual / quarterly report.
- **/A suffix** — amendment (e.g., 10-K/A).
- **Instant vs. duration** — XBRL distinction between point-in-time facts (balance sheet) and over-period facts (income / cash flow).
- **Fiscal period** — a company-defined accounting period; may not align with the calendar year.
- **Item 4.02 8-K** — a Form 8-K filing under Item 4.02, "Non-Reliance on Previously Issued Financial Statements." The SEC requires this filing when management concludes that previously filed financial statements should no longer be relied upon.
- **Restated** — corrected version of a previously filed financial statement, typically published in a 10-K/A or 10-Q/A amendment.
- **52/53-week fiscal year** — a fiscal calendar (common for retailers) where each year ends on a fixed weekday, producing a 52- or 53-week year depending on calendar drift.
- **MD&A** — Management's Discussion and Analysis, the prose section of a 10-K/10-Q where management explains the financial results. Out of V1 scope as a content source.

C. Document conventions

- Code blocks marked `// Sketch` are illustrative, not final API.
- Schema DDL in §6.3 is the canonical baseline for migration `0001_initial.sql`.
- Any deviation from the catalog or DDL during implementation must be captured in a follow-up ADR under `docs/adr/`.